AuraBot AI

Complete Professional AI Assistant
Project Blueprint

Website + Android App | ChatGPT-style Experience | Advanced AI Assistant

GDD / Product Design • Features • Architecture • Database • APIs • UI/UX • Roadmap

Prepared as a practical development guide
Version 1.0 | 05 November 2026

Project vision: AuraBot AI ek secure, multilingual, tool-enabled assistant hoga jo natural conversation, document analysis, web research, voice interaction aur user-authorized automation support karega.

Important scope note: Ye document complete product blueprint hai, ready-to-deploy source-code package nahi. Actual implementation ke dauran AI provider, hosting, API keys, privacy policy, budget aur platform release requirements configure karne honge.

Contents

- 1. Executive Summary & Product Goals
- 2. Product Scope and User Personas
- 3. Complete Feature Catalogue
- 4. User Journeys and Working Process
- 5. UI/UX Design Specification
- 6. Technical Architecture
- 7. Recommended Technology Stack
- 8. Project Folder Structure
- 9. Database Design
- 10. API Specification
- 11. AI Orchestration and Tool System
- 12. Security, Privacy and Safety
- 13. Testing and Quality Plan
- 14. Deployment and Operations
- 15. 14-Week Development Roadmap
- 16. MVP Acceptance Criteria
- 17. Risks, Cost Controls and Future Scope
- 18. Glossary and Build Checklist

How to use this document: Pehle MVP scope aur architecture lock karein. Uske baad Phase 1 se coding start karein. Har phase ke acceptance criteria pass hone ke baad hi next phase par move karein.

1. Executive Summary & Product Goals

AuraBot AI ek multi-platform personal AI assistant hoga. Website aur Android app same backend, same account system aur shared conversation history use karenge.

Primary goals

- Natural language conversation: Hindi, Hinglish aur English mein user input samajhna.
- Contextual assistance: current conversation ke relevant context ko use karke coherent answers dena.
- Tool use: permission ke saath web search, file processing, calculations aur connected services use karna.
- Multi-modal experience: text, voice, images aur documents ko support karna.
- Cross-platform continuity: website par shuru hui chat Android par continue ho sake.
- Privacy and control: user apni chats, files, memories aur connected services manage kar sake.

Product principles

Principle	Product rule
Useful first	Har feature ka clear user benefit ho; unnecessary complexity avoid karein.
Transparent AI	Jab answer uncertain ho ya external source use ho, limitation/source bataye.
User control	Memory, file retention, integrations aur automation user ke control mein hon.
Secure by default	Least privilege, server-side secrets, tenant isolation and auditability.
Accessible	Keyboard navigation, readable contrast, mobile-friendly controls.
Incremental delivery	Pehle reliable MVP, phir advanced capabilities.

Success metrics (initial targets)

Metric	Initial target / measurement
Chat reliability	Successful response completion rate; errors tracked by type.
Latency	Time to first token and total response time measured by model and task.
User experience	Feedback rating, repeat usage, conversation completion.
Safety	Unauthorized tool execution target: zero; policy incidents reviewed.
Cost	Track token, search, storage and transcription cost per active user.
Stability	Crash-free sessions and API uptime tracked after launch.

2. Product Scope and User Personas

Target users

Persona	Needs	Typical use
Student / learner	Explanations, notes, language help, study support	Ask a concept, upload notes, generate a revision summary.
Office / business user	Documents, spreadsheets, email drafts, reminders	Analyze a workbook, draft a message, organize tasks.
Developer / creator	Code explanations, debugging, brainstorming	Ask for code, understand an error, plan a feature.
General user	Everyday questions, research, voice interaction	Ask questions in Hindi/Hinglish, search current information.

In-scope for planned product

- Chat, chat history, account settings and cross-device sync.
- AI model integration through a secure backend.
- File upload and supported document extraction.
- Optional web search integration with citations.
- Voice input/output using supported browser or provider services.
- User-controlled memory and limited tool integrations.
- Admin monitoring for service health, abuse and usage.

Out-of-scope for first release

- Training a foundation model from scratch.
- Unrestricted autonomous access to a user's device or accounts.
- Guaranteed factual correctness without verification.
- High-risk decisions without human review.
- Unlimited free AI usage; model and hosting costs must be controlled.

MVP definition

MVP = responsive website + login + chat + one LLM provider + streaming response + conversation history + basic settings + usage limits. Android wrapper and advanced tools can follow after the website MVP is stable.

3. Complete Feature Catalogue

A. Smart Chat Engine

- Create, rename, archive and delete conversations.
- Streaming response with stop-generation control.
- Edit user message and regenerate assistant answer.
- Copy response, share link (optional) and give feedback.
- Markdown, code blocks, tables and safe rich-text rendering.
- Conversation search and pagination.
- Retry after transient errors; clear error messages.

B. Voice Engine

- Push-to-talk voice input as first release option.
- Speech-to-text with language selection where supported.
- Text-to-speech playback, pause and stop.
- Optional real-time voice conversation in later phase.
- Microphone permission explanation and visible recording state.
- Do not retain raw audio longer than necessary; disclose provider retention.

C. Memory Engine

- Short-term context from current conversation.
- Optional saved facts/preferences with explicit user controls.
- Memory review, edit, disable and delete controls.
- Memory source and date metadata.
- Per-user memory isolation; no cross-account retrieval.
- Avoid storing sensitive details unless essential and explicitly requested.

D. File & Image Engine

- Upload PDF, DOCX, XLSX, CSV, TXT and supported image formats.
- File size/type validation and malware scanning.
- Text extraction and spreadsheet parsing.
- Summarize and ask questions about uploaded content.
- Show source filename and page/sheet references when possible.
- Temporary processing files and configurable retention/deletion.

E. Web Research Engine

- Search provider integration behind backend adapter.
- Search current information when user asks or task requires it.
- Summarize and cite source URLs and access dates.
- Separate sourced facts from model-generated interpretation.
- Handle failed, blocked or stale sources transparently.

F. Automation & Integrations

- Reminders and task records.
- Google Sheets, Calendar and email integrations through OAuth or scoped credentials.
- Preview action and request confirmation before external side effects.
- Show action result and allow cancellation where supported.
- Per-integration disconnect and permission management.

G. Coding Assistant

- Explain code and errors.
- Generate code snippets and project scaffolds.
- Support syntax highlighting and copy action.
- Never execute untrusted code directly on the application server.
- Sandbox any future code execution with strict limits.

H. Data Analysis

- CSV/XLSX parsing and data preview.
- Data cleaning suggestions and duplicate detection.
- Formula explanation and generation.
- Charts and downloadable report generation.
- Show assumptions, units and calculation method.

I. User & Account

- Signup, login, logout and password reset.
- Profile, language and theme settings.
- Session management and device logout.
- Account export and deletion workflow.
- Usage quota and plan information.

J. Admin & Operations

- User and account status management with role controls.
- Usage, token cost, latency and error dashboards.
- Provider/model configuration through secure server settings.
- Abuse detection and rate-limit controls.
- Audit trail for administrative actions.
- No admin access to private chat content by default; restricted audited access only when legally and operationally necessary.

Feature prioritization

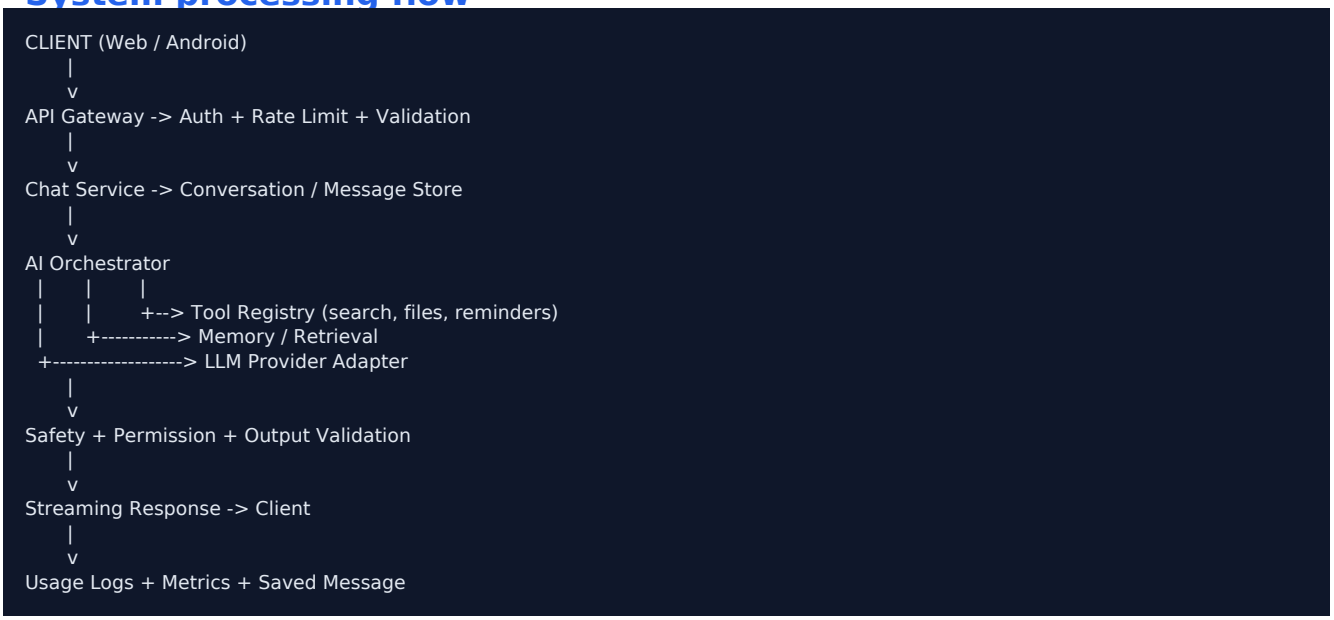
Priority	Features	Release
P0 – Must have	Chat, authentication, streaming, history, settings, usage limits, security	MVP 1.0
P1 – Should have	File upload, memory controls, web search, feedback, mobile packaging	2.0
P2 – Advanced	Voice conversation, data analysis, integrations, admin analytics	3.0
P3 – Later	Team workspaces, custom assistants, advanced retrieval, subscriptions	4.0+

4. User Journeys and Working Process

Primary chat journey

1. User website ya Android app open karta hai.
2. User login karta hai ya allowed guest mode use karta hai.
3. New chat create hoti hai; user text, voice ya supported attachment bhejta hai.
4. Client request backend ko authenticated session ke saath bhejta hai.
5. Backend input validate karta hai, conversation ownership verify karta hai aur quota check karta hai.
6. Orchestrator relevant context retrieve karta hai aur model/tool decision leta hai.
7. Tool call ho to permission and confirmation rules apply hote hain.
8. AI response stream hota hai; final message database mein save hota hai.
9. User response copy, rate, regenerate ya follow-up kar sakta hai.

System processing flow



Example: PDF summary

1. User PDF attach karta hai aur summary request karta hai.
2. Backend file extension ke saath MIME type, size aur content validate karta hai.
3. File malware scan / safe processing pipeline se pass hoti hai.
4. PDF text extraction hota hai; scanned PDF ke liye OCR optional hai.
5. Relevant text chunks model ko bheje jaate hain; very large documents ke liye retrieval use hota hai.
6. Assistant concise summary, key points aur page references deta hai.
7. Temporary artifacts retention policy ke according delete hote hain.

Example: external action

Agar user kahe 'calendar mein meeting add karo', assistant pehle date/time aur title clarify karega, action preview dikhayega aur user confirmation ke baad hi calendar API call karega. Successful action ka result aur event reference show karega.

5. UI/UX Design Specification

Design direction

Visual style: clean, calm, modern SaaS interface. Dark navy/charcoal base, restrained blue accent, clear typography, spacious chat area. Avoid excessive gradients, animations or visual clutter.

Desktop layout

Region	Contents
Left sidebar	AuraBot logo, New Chat, search chats, history list, Explore, Files, Settings, profile.
Main header	Current chat title, model selector (if enabled), share/export controls.
Conversation area	User and assistant messages, timestamps, citations, code/table rendering, loading state.
Composer	Multiline input, attach file, voice input, send, stop generation, usage notice.
Optional right panel	File details, citations, memory references or tool activity when relevant.

Mobile layout

- Sidebar becomes a drawer or bottom navigation depending on screen.
- Composer remains reachable above keyboard and respects safe areas.
- Buttons have touch-friendly hit areas.
- File picker, microphone and share sheet use native Android capabilities.
- Long responses, tables and code blocks must scroll horizontally or reflow safely.

Core screens

Screen	Key components
Welcome / Home	Brand, prompt suggestions, recent chats, input composer.
Chat	Messages, composer, streaming state, citations, feedback actions.
History	Search, sort, rename, archive and delete.
Explore	Available assistant tools and feature descriptions.
Files	Uploaded file list, status, preview and delete.
Memory	Saved memory list, edit, disable and delete.
Settings	Theme, language, privacy, account, connected apps.
Admin	Usage, system health, moderation and configuration.

UX states that must be designed

- Empty state: show examples and a clear first action.
- Loading state: show that the assistant is working; allow stop where possible.
- Error state: explain what failed and offer retry where safe.
- Offline state: explain that cloud AI features need connectivity.
- Quota state: show remaining usage or reset details.
- Permission state: explain why microphone/file/integration access is needed.
- Destructive state: confirm delete account, chat, file or memory.

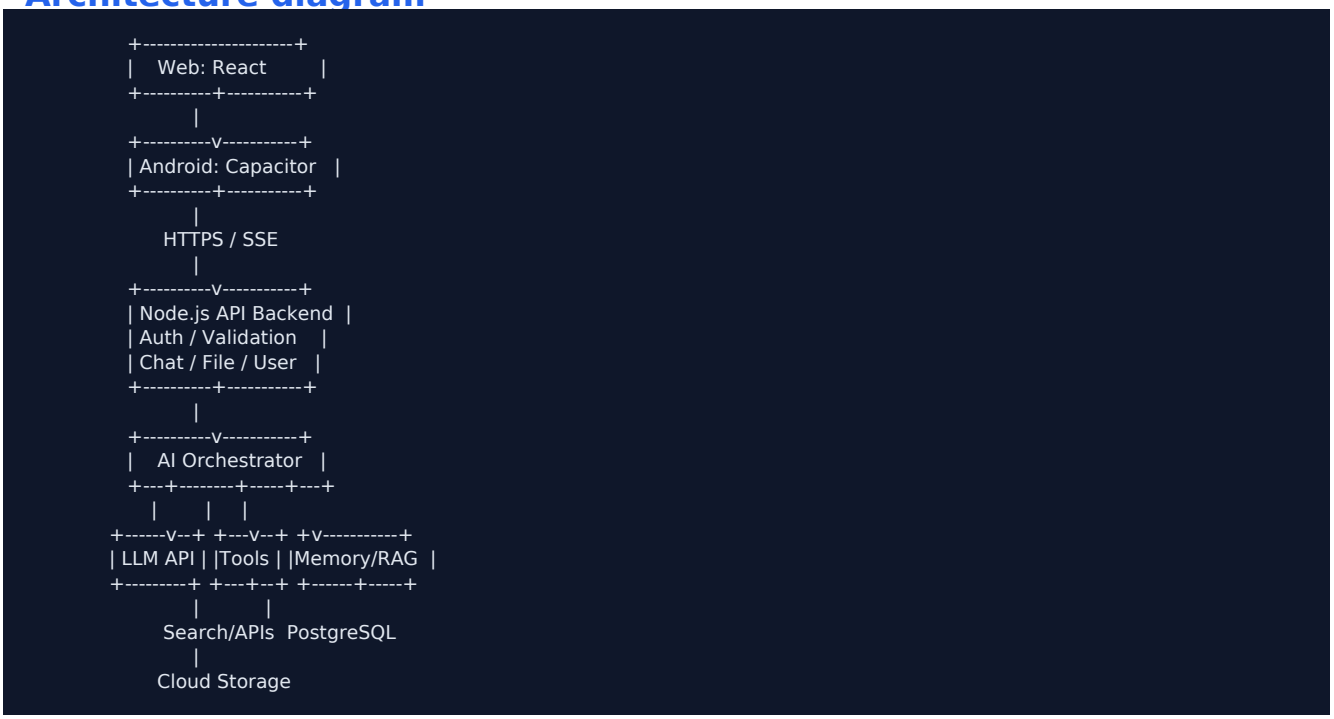
6. Technical Architecture

Recommended architecture: modular monolith for the first release. Keep services separated by domain inside one backend deployment. Split into microservices only when traffic, team size or independent scaling justifies it.

High-level components

Component	Responsibility
Web client	React UI, session handling, chat streaming and responsive layout.
Android client	Capacitor shell, native permissions, file picker and notifications.
API gateway / backend	Authentication, authorization, validation, rate limits and routing.
Chat service	Conversation lifecycle, message persistence and streaming orchestration.
AI orchestrator	Prompt assembly, model routing, tool selection, retries and output handling.
Tool registry	Allow-listed tools with schemas, permission levels and timeouts.
Memory service	User-controlled memory storage and retrieval.
File service	Upload, scanning, extraction, indexing and deletion.
Database	Users, chats, messages, metadata, usage and audit events.
Observability	Metrics, traces, error reporting and cost monitoring.

Architecture diagram



Communication choices

- REST APIs for account, history, files, settings and CRUD operations.
- Server-Sent Events (SSE) for token-by-token text streaming in the first version.
- WebSocket only if real-time bidirectional voice or collaboration requires it.
- Background job queue for long file processing, indexing, exports and scheduled tasks.

Deployment topology

Start with one frontend hosting target, one backend service, managed PostgreSQL and object storage. Keep secrets in a managed secret store or protected environment configuration. Use separate development, staging and production environments.

7. Recommended Technology Stack

Layer	Recommendation	Reason / Notes
Frontend	React + TypeScript	Reusable components and typed application logic.
Build tool	Vite	Fast local development and production build.
Styling	Tailwind CSS	Consistent responsive UI.
Mobile	Capacitor	Reuse web UI while accessing selected native features.
Backend	Node.js + Express	Common TypeScript ecosystem; simple API service.
Validation	Zod or equivalent	Validate request and tool schemas.
API style	REST + SSE	Straightforward CRUD and streamed responses.
Database	PostgreSQL	Relational data, transactions and mature tooling.
Vector retrieval	pgvector	Optional embeddings and document retrieval.
Authentication	Firebase Auth or managed OIDC	Avoid writing password/session infrastructure from scratch.
File storage	S3-compatible object storage	Private buckets, signed URLs and lifecycle policies.
AI provider	Provider adapter around LLM API	Allows model/provider changes without rewriting app.
Monitoring	Error tracker + metrics	Track latency, failures, token usage and cost.
CI/CD	GitHub Actions or equivalent	Automated tests and deployment workflow.

Provider abstraction

Backend mein AIProvider interface define karein: generate(), stream(), embed() (if used), and supported model metadata. OpenAI, Gemini ya another provider adapter is interface ko implement kare. Isse provider switch karna easier hoga. Provider-specific features ko common interface mein force na karein; capability flags use karein.

Windows 10 development note

8 GB RAM aur Intel Core i3 par React/Node development possible hai, lekin Android Studio emulator heavy ho sakta hai. Initial work VS Code + browser + cloud AI API se karein. Android testing ke liye physical Android phone use karna lighter option ho sakta hai.

8. Project Folder Structure

```
AuraBot-AI/
|
|-- apps/
|   |-- web/
|   |   |-- public/
|   |   |-- src/
|   |   |   |-- app/
|   |   |   |-- components/
|   |   |   |-- pages/
|   |   |   |-- features/
|   |   |   |   |-- chat/
|   |   |   |   |-- files/
|   |   |   |   |-- memory/
|   |   |   |   |-- settings/
|   |   |   |-- services/
|   |   |   |-- hooks/
|   |   |   |-- styles/
|   |   |   |-- types/
|   |   |   |   `-- main.tsx
|   |   |   `-- package.json
|   |   |
|   |-- mobile/
|   |   |-- capacitor.config.ts
|   |   `-- android/
|   |
|   `-- api/
|       |-- src/
|       |   |-- config/
|       |   |-- routes/
|       |   |-- controllers/
|       |   |-- middleware/
|       |   |-- modules/
|       |   |   |-- auth/
|       |   |   |-- chat/
|       |   |   |-- users/
|       |   |   |-- files/
|       |   |   |-- memory/
|       |   |   |-- search/
|       |   |   |-- integrations/
|       |   |   `-- admin/
|       |   |-- ai/
|       |   |   |-- orchestrator/
|       |   |   |-- providers/
|       |   |   |-- prompts/
|       |   |   `-- tools/
|       |   |-- jobs/
|       |   |-- database/
|       |   |-- validators/
|       |   `-- server.ts
|       `-- package.json
|
|-- packages/
|   |-- shared-types/
|   |-- ui/
|   `-- config/
|
|-- database/
|   |-- migrations/
|   |-- seeds/
|   `-- schema.sql
|
|-- docs/
|   |-- PRODUCT_BLUEPRINT.md
|   |-- API_SPEC.md
|   |-- SECURITY.md
|   `-- RUNBOOK.md
|
|-- .env.example
|-- .gitignore
|-- package.json
`-- README.md
```

Folder responsibilities

Path	Purpose
apps/web	Browser application.
apps/mobile	Capacitor configuration and Android project.
apps/api	Backend application and API routes.
apps/api/src/ai	Provider adapters, prompts, orchestrator and tools.
packages/shared-types	Types shared by web and API.
database/migrations	Version-controlled schema changes.
docs	Product, API, security and operations documentation.

Keep environment-specific secrets out of Git. Commit .env.example with variable names and safe placeholders only.

9. Database Design

Core relational tables

Table	Important columns	Notes
users	id, auth_subject, email, display_name, created_at, status	auth_subject maps to managed identity provider.
conversations	id, user_id, title, status, created_at, updated_at	user_id foreign key; owner-scoped.
messages	id, conversation_id, role, content, model, status, created_at	Store structured content safely; index conversation/time.
memories	id, user_id, memory_text, category, source, enabled, created_at	User review/edit/delete required.
files	id, user_id, storage_key, filename, mime_type, size, status, created_at	Private object storage; never trust filename.
file_chunks	id, file_id, chunk_index, text, embedding	Optional RAG; enforce file ownership.
reminders	id, user_id, title, due_at, status, created_at	Timezone-aware timestamps.
integrations	id, user_id, provider, token_ref, scopes, status	Store token references securely, not plaintext tokens.
usage_logs	id, user_id, model, input_tokens, output_tokens, cost_estimate, created_at	Cost and quota tracking.
feedback	id, user_id, message_id, rating, comment, created_at	Avoid collecting unnecessary personal details.
audit_logs	id, actor_id, action, target_type, target_id, created_at	Metadata only; restrict access.

Relationship summary

- One user has many conversations, files, memories, reminders and integrations.
- One conversation has many messages.
- One file may have many extracted chunks.
- Feedback belongs to a message and user.
- All user-owned records must be checked against authenticated user ID on every request.

Database rules

- Use UUID primary keys or another non-guessable identifier.
- Add foreign keys and appropriate indexes.
- Use migrations; do not edit production schema manually.
- Use transactions for multi-step writes.
- Use row-level security where supported, plus application authorization checks.
- Encrypt backups and define retention/deletion policies.

10. API Specification

Base URL examples: /api/v1. All private routes require a valid access token. The exact authentication mechanism depends on the selected identity provider.

Method + endpoint	Purpose	Auth
POST /auth/session	Exchange/verify sign-in session	Public/Provider
GET /me	Current user profile	Yes
PATCH /me	Update profile/settings	Yes
POST /chats	Create conversation	Yes
GET /chats	List user's conversations	Yes
GET /chats/:chatId	Read one conversation	Yes
PATCH /chats/:chatId	Rename/archive conversation	Yes
DELETE /chats/:chatId	Delete conversation	Yes
POST /chats/:chatId/messages	Send message; stream response	Yes
POST /files	Upload file and create processing job	Yes
GET /files	List user's files	Yes
DELETE /files/:fileId	Delete file and related chunks	Yes
GET /memories	List saved memories	Yes
POST /memories	Create a memory	Yes
PATCH /memories/:id	Edit/disable memory	Yes
DELETE /memories/:id	Delete memory	Yes
POST /search	Run approved web search	Yes
POST /reminders	Create reminder	Yes
GET /usage	Read usage and quota	Yes
POST /feedback	Rate an assistant response	Yes

Chat request example

```
POST /api/v1/chats/{chatId}/messages
Authorization: Bearer <access_token>
Content-Type: application/json

{
  "content": "Explain this spreadsheet",
  "attachments": [],
  "stream": true
}
```

Streaming response

Use SSE events such as message_start, token, tool_status, citation, message_complete and error. The client must handle disconnects, retry safely and avoid duplicating a message on reconnect.

API design rules

- Validate body, query and path parameters.
- Use consistent error shape: code, message, request_id.

- Use pagination for history and file lists.
- Apply ownership checks before reading or modifying resources.
- Use idempotency keys for actions that may be retried.
- Set request size, timeout and rate limits.

11. AI Orchestration and Tool System

The orchestrator is the control layer between the user request, AI model and approved tools.

Orchestration steps

1. Normalize user input and detect language/task type.
2. Load only relevant recent conversation context.
3. Retrieve enabled, relevant user memories if the user has opted in.
4. Choose model based on task, capability, cost and latency policy.
5. Provide the model with available tool schemas and permission boundaries.
6. If a tool is requested, validate arguments server-side.
7. For side-effect actions, show a preview and obtain explicit confirmation.
8. Execute the tool with timeout, least privilege and audit metadata.
9. Pass tool result back to the model for a user-facing explanation.
10. Validate and stream final response; log usage and errors.

Tool registry design

Tool	Input example	Permission / control
web_search	query, recency, domains	Read-only; cite sources.
read_uploaded_file	file_id, page/sheet range	Must belong to user; scan and validate.
calculate	expression or structured data	Validate; no arbitrary code execution.
create_reminder	title, due_at, timezone	Confirm details before saving if ambiguous.
create_calendar_event	title, start, end, attendees	Preview and explicit confirmation.
draft_email	recipient, subject, body	Draft only by default; sending requires confirmation.
update_google_sheet	spreadsheet, range, values	Scoped OAuth; preview changes and confirm.

Memory strategy

- Short-term: current chat messages and a bounded recent context window.
- Long-term: separate memory records controlled by the user.
- Retrieval: select only relevant memories; do not inject the entire memory store.
- Source metadata: store when/why a memory was created where appropriate.
- Controls: review, edit, disable, delete and clear memory options.
- Sensitive information: avoid saving sensitive data by default.

Retrieval-Augmented Generation (RAG)

For long documents, split extracted text into chunks, attach document/page metadata, optionally create embeddings and retrieve relevant chunks for each question. Always enforce user ownership at retrieval time. RAG can improve relevance but does not guarantee correctness; provide references back to the source document.

12. Security, Privacy and Safety

Security baseline

- TLS/HTTPS for all network traffic.
- Managed identity provider or securely implemented authentication.
- Never place AI API keys, database passwords or OAuth client secrets in frontend/mobile bundles.
- Server-side authorization on every resource access.
- Tenant isolation: every query is scoped to authenticated user or authorized workspace.
- Rate limits by user, IP and expensive operation.
- Upload allow-list, MIME inspection, size limit, malware scanning and safe parsing.
- Secure cookies/token storage; avoid long-lived tokens in insecure browser storage.
- Dependency scanning, secret scanning and timely updates.
- Encrypted backups and tested restore process.

AI-specific safeguards

- Treat uploaded documents and web pages as untrusted data, not system instructions.
- Tool allow-list: model cannot invent arbitrary tools or call arbitrary URLs.
- Validate tool arguments against strict schemas.
- Require confirmation for external side effects such as sending, deleting or modifying records.
- Use timeouts, quotas and sandboxing for risky processing.
- Do not execute arbitrary generated code on the production server.
- Cite external sources when web research is used; disclose uncertainty.

Privacy controls

Data type	Recommended control
Chat history	User can delete chats; define retention and backup deletion window.
Memory	Opt-in/transparent controls; review, edit, disable and delete.
Uploaded files	Private storage, signed short-lived access, retention and deletion.
Voice	Explain microphone use and provider processing; retain audio only if needed.
Integrations	OAuth scopes limited to needed actions; disconnect and revoke.
Analytics	Minimize personal content; aggregate where possible.

Operational policy

Publish clear Terms, Privacy Policy, acceptable-use rules, data retention policy and support contact before public launch. Define an incident response process and a way for users to report harmful or incorrect output.

13. Testing and Quality Plan

Test level	What to test	Examples
Unit	Individual functions and services	Prompt builder, validators, quota calculator.
Integration	Service-to-service behavior	Chat service + database; file parser + storage.
API	Status, schema, auth and errors	Unauthenticated request denied; wrong owner denied.
UI	User workflows and responsive layout	New chat, send, stop, retry, upload, delete.
AI evaluation	Quality and consistency	Known test prompts, citation checks, refusal behavior.
Security	Common vulnerabilities and access control	IDOR, injection, XSS, upload abuse, rate limits.
Performance	Latency and concurrency	Streaming first token, large file processing.
Mobile	Device permissions and lifecycle	Microphone, picker, keyboard, offline/reconnect.

Release checklist

- Critical user journeys pass on Chrome desktop and Android phone.
- No API secrets in source maps, app bundle or repository.
- Users cannot access another user's chats/files/memories.
- File size/type limits and error handling are tested.
- Tool actions require the right permission and confirmation.
- Backups and restore procedure are verified.
- Privacy policy, terms, support contact and deletion process are available.
- Usage alerts and cost limits are configured.

14. Deployment and Operations

Environment separation

Environment	Purpose	Data
Local	Developer machine	Synthetic/test data only.
Development	Shared feature testing	Non-production test accounts.
Staging	Release rehearsal	Production-like configuration, test data.
Production	Real users	Protected data, backups and monitoring.

Deployment sequence

1. Create production project and managed database.
2. Configure environment variables and secrets in hosting provider.
3. Run database migrations and create least-privilege database user.
4. Deploy backend and health-check endpoint.
5. Deploy frontend with correct API base URL.
6. Configure HTTPS, domain, CORS and allowed origins.
7. Test login, chat streaming, file upload, deletion and rate limits.
8. Build Android package using Capacitor; test on physical devices.
9. Set monitoring alerts, backups and rollback procedure.

Environment variables example (names only)

```
NODE_ENV=production
PORT=3000
DATABASE_URL=<managed database connection>
AUTH_ISSUER=<identity provider issuer>
AUTH_AUDIENCE=<api audience>
AI_PROVIDER=<provider name>
AI_API_KEY=<server-side secret>
OBJECT_STORAGE_BUCKET=<private bucket>
SEARCH_API_KEY=<server-side secret>
APP_ORIGIN=<website origin>
```

Do not commit real values. Keep .env out of version control. Rotate any key that is accidentally exposed.

Operations to monitor

- API availability and error rate.
- Model latency, token usage and cost per user/task.
- File processing queue depth and failures.
- Database CPU, storage, connections and backup status.
- Abuse signals, authentication failures and rate-limit events.
- Android crash reports and app version adoption.

15. 14-Week Development Roadmap

Week	Focus	Deliverables / exit criteria
1	Project foundation	Git repo, Node.js, React/TS/Vite, linting, environment config, base routing.
2	UI foundation	Responsive shell, sidebar, chat page, composer, theme and mobile layout.
3	Backend foundation	Express API, health check, request validation, auth integration.
4	AI chat	LLM adapter, secure key handling, streaming, stop/retry and usage logging.
5	Database & history	PostgreSQL schema, migrations, conversations, messages, ownership checks.
6	File processing	Upload, private storage, validation, PDF/text extraction and deletion.
7	Search & memory	Web search adapter, citations, memory review and retrieval controls.
8	Voice	Speech input/output prototype, permissions, accessibility and fallback.
9	Tools	Tool registry, schemas, confirmation flow, reminders and safe calculations.
10	Analytics & integrations	CSV/XLSX analysis, report output, one scoped external integration.
11	Android packaging	Capacitor, Android build, file picker, keyboard and permission tests.
12	Cross-platform polish	Sync, reconnect behavior, notifications (if needed), responsive QA.
13	Security & performance	Threat review, access tests, rate limits, load tests, cost controls.
14	Launch preparation	Staging sign-off, docs, policies, production deploy, Android release candidate.

Weekly working rhythm

- Plan: define one small, testable outcome for the week.
- Build: implement only the scoped features.
- Test: run unit, integration and manual checks.
- Review: check UX, security, accessibility and cost.
- Document: update setup, API and known limitations.

Time estimate is a planning baseline, not a guarantee. A beginner working part-time may need longer, especially for authentication, Android packaging, security and deployment.

16. MVP Acceptance Criteria

Version 1.0 is considered ready when all of these are true:

- User can sign in and sign out securely.
- User can create, rename and delete own conversations.
- User can send a message and receive a streamed AI response.
- User can stop generation and retry after a recoverable failure.
- Conversation persists and reloads correctly.
- A user cannot read or change another user's data.
- API key is never exposed to browser or Android client.
- Rate limits and basic usage tracking are active.
- Desktop and mobile web layouts are usable.
- Errors are understandable and do not expose secrets or stack traces.
- Basic monitoring and backup are configured.

Definition of done for each feature

A feature is done only when implementation, validation, error states, authorization, tests, documentation and user-facing behavior are complete. A button that looks interactive but has no working action is not considered complete.

17. Risks, Cost Controls and Future Scope

Main project risks

Risk	Mitigation
AI API cost rises	Set per-user quotas, model routing, token caps, caching where safe and alerts.
Incorrect AI answers	Use source citations, file references, validation and uncertainty disclosure.
Private data exposure	Tenant isolation, scoped access, encryption and security testing.
Prompt injection in files/web	Treat content as untrusted; tools are allow-listed and permission-checked.
Slow file processing	Background jobs, size limits, progress status and cancellation.
Android performance limits	Reuse web app, test on physical devices, optimize bundle and images.
Provider outage	Graceful errors, retry with backoff and optional provider adapter fallback.

Cost categories

- LLM API: input/output tokens, model tier and tool calls.
- Hosting: frontend, backend compute and bandwidth.
- Database: storage, compute, backups and connections.
- File storage: upload volume, downloads and retention.
- Search/voice: third-party request pricing.
- Domain and optional app distribution fees.

Cost-control actions

- Start with a low-cost model for routine tasks and use stronger models only when justified.
- Limit maximum context and file size.
- Track estimated cost per request and user.
- Set monthly spend alerts and hard usage quotas where possible.
- Use asynchronous jobs for expensive document processing.
- Offer clear usage limits rather than promising unlimited service.

Future scope after stable launch

- Custom assistants and saved instructions.
- Team workspaces and shared knowledge bases.
- Subscriptions and billing.
- More languages and voice options.
- Advanced RAG and document collections.
- Admin analytics and organization controls.
- Native Android enhancements if Capacitor limits become significant.

18. Glossary and Final Build Checklist

Term	Meaning
LLM	Large Language Model; AI model that generates and interprets language.
API	Application Programming Interface; defined way for systems to communicate.
SSE	Server-Sent Events; server streams updates to a client over HTTP.
RAG	Retrieval-Augmented Generation; retrieves relevant source material for an AI answer.
Embedding	Numeric representation of text used for semantic similarity search.
Orchestrator	Backend component coordinating model calls, context and tools.
OAuth	Authorization standard used to grant scoped access to external services.
MVP	Minimum Viable Product; smallest useful release that can be tested by users.
CI/CD	Automated testing and delivery pipeline.
Tenant isolation	Ensuring each user or organization can access only its own data.

Final project checklist

- [ ] Product scope and MVP approved.
- [ ] Repository and development environment ready.
- [ ] Frontend shell and responsive chat UI completed.
- [ ] Backend, authentication and database connected.
- [ ] AI provider adapter and streaming work.
- [ ] Chat history and ownership checks pass.
- [ ] File pipeline and search work within limits.
- [ ] Memory controls and privacy policy implemented.
- [ ] Tool confirmation and audit logging work.
- [ ] Android package tested on physical device.
- [ ] Security, performance and backup checks pass.
- [ ] Production launch and support process ready.

Final recommendation

AuraBot AI ko ek hi baar mein full ChatGPT-scale product banane ki koshish na karein. Pehle reliable text-chat MVP ship karein. Uske baad files, memory, web search, voice, automation aur Android packaging ko measurable milestones mein add karein. Is approach se cost, complexity aur debugging risk control mein rahega.

Document version: 1.0 | Prepared: 05 November 2026 | Project: AuraBot AI